OPENCLAW → ALLUCI

Feature Gap Analysis & Integration Roadmap

Itemised catalogue of capabilities present in OpenClaw that are absent from Alluci-Sovereign-Agent,
with priority ratings, effort estimates, and implementation guidance.

Communication Channels	**8 fully-wired channels (WhatsApp, Telegram, Discord, Slack, Signal, iMessage, Nostr, Google Chat)**	**Adapters stubbed — no real integration**
Scheduling	**Cron job engine with 3 schedule types, delivery routing, run history**	**No scheduler**
Usage & Cost Analytics	**Date-range dashboards, per-session costs, CSV export, time-series charts**	**No analytics**
Admin Control UI	**13-tab web dashboard via WebSocket gateway**	**No dashboard**
Skill Registry	**Install, enable/disable, API-key management**	**Regex-only guardrail stubs**
Device / Node Pairing	**Signed device identity, QR pairing, node binding**	**WebAuthn stub only**
Config Management	**Schema-driven live editor with hot-apply**	**No live config editor**
Log Streaming	**JSONL tail, level filters, auto-follow, export**	**No log viewer**

Prepared March 2026 · Based on full source analysis of both repositories

1 Executive Summary

OpenClaw is a production-grade, multi-channel AI gateway with a sophisticated control plane. Alluci-Sovereign-Agent shares the same high-level ambition — a locally-run sovereign AI assistant — but its non-core systems (channels, scheduler, analytics, administration) are largely scaffolded stubs rather than working implementations.

This document catalogues every feature or capability found in OpenClaw that is absent or non-functional in Alluci. Items are organised by domain, rated for integration priority, and accompanied by implementation notes. The goal is not to clone OpenClaw wholesale, but to identify the patterns and sub-systems Alluci should adopt to reach production readiness.

Overall gap verdict

OpenClaw is approximately 18–24 months ahead of Alluci in operational maturity. The core AI logic in Alluci (VaultManager, ModelRouter, Planner DAG, AuditLedger) is sound. The surrounding infrastructure — channels, scheduling, observability, administration — needs to be built from scratch, and OpenClaw provides an excellent reference architecture for each of those areas.

Gap Count by Domain

Communication Channels	12	4	5	2	1
Scheduling & Automation	8	2	4	2	0
Usage & Cost Analytics	9	1	4	3	1
Administration & Control UI	11	2	5	3	1
Skill / Plugin Registry	6	1	3	2	0
Device & Node Management	5	2	2	1	0
Config Management	4	1	2	1	0
Log Streaming & Observability	4	0	3	1	0
Chat UX	7	1	3	2	1
Security & Auth Infrastructure	5	3	2	0	0
TOTAL	71	17	33	17	4

2 Communication Channels

OpenClaw ships 8 fully-implemented, production-ready channel adapters that handle authentication, health monitoring, per-account management, and UI configuration cards. Alluci's bridge_adapters folder contains empty scaffolding for a subset of these.

Alluci status

No channel adapter in Alluci has working send/receive logic. The Telegram adapter imports Telethon but never creates a session. WhatsApp imports whatsapp-web.js but the class body is empty. Signal and Discord are not present at all.

2.1 WhatsApp — QR-Code Pairing Flow

OpenClaw renders a live QR code image in the UI when a WhatsApp session is initialising. The QR data is streamed from the gateway over WebSocket and displayed as a base64 PNG. Alluci has no pairing flow, no QR display, and no real whatsapp-web.js session management.

- Implement a QR code endpoint that streams base64 QR data from the whatsapp-web.js client.ready event.
- Add a pairing state machine: IDLE → QR_PENDING → CONNECTED → DISCONNECTED.
- Expose pairing status in the health endpoint so the frontend can show current state.
- Store session state on disk so connections survive restarts.

2.2 Telegram — Bot + Webhook Management

OpenClaw stores the bot token, displays the webhook URL, shows a linked account count, and reports the bot's last activity. Alluci's TelegramBridge constructor body is empty.

- Implement bot token validation against the Telegram Bot API on startup.
- Auto-register a webhook URL using setWebhook; expose the URL in the admin interface.
- Track connected accounts and expose a per-account list with last-seen timestamps.
- Implement inbound message parsing: text, photo, document, voice, sticker types.

2.3 Discord — Guild / Channel Selectors

OpenClaw supports Discord bot token configuration, guild browsing, and per-channel message routing. Alluci has no Discord adapter.

- Implement discord.js client with bot token auth and guild/channel enumeration.
- Store guild-to-channel mapping in the database for routing logic.
- Implement slash command registration for the guild.

2.4 Slack — OAuth App Configuration

OpenClaw handles Slack OAuth app credentials (client ID, client secret, signing secret), workspace count reporting, and message event subscriptions. Alluci has no Slack adapter.

- Implement OAuth 2.0 install flow and token storage.
- Handle Events API (URL verification challenge + message.channels events).

- Implement Block Kit message formatting for rich responses.

2.5 Signal — Phone Number Registration

OpenClaw provides phone-number-based Signal registration through signal-cli. Alluci has no Signal integration.

- Integrate signal-cli as a subprocess adapter.
- Implement registration flow: phone number → SMS/voice verification code → link device.
- Handle group messaging and attachments.

2.6 iMessage — macOS-Only Adapter

OpenClaw renders an iMessage card that correctly shows a platform restriction callout on non-macOS systems. Alluci has no iMessage adapter.

- Implement macOS-only adapter using AppleScript / osascript for send/receive.
- Add platform detection on startup; gracefully disable the adapter on non-macOS with a clear status message in the health endpoint.

2.7 Nostr — Decentralised Protocol

OpenClaw implements a full Nostr identity profile (display name, about, picture URL, banner URL, NIP-05, LUD-16) and relays communication through configurable relays. Alluci has no Nostr support.

- Generate or import a Nostr keypair (nsec / npub).
- Implement NIP-01 (basic protocol) event publishing and subscription.
- Implement NIP-04 (encrypted DMs) for private messaging.
- Implement NIP-05 identity verification endpoint.

2.8 Google Chat — Service Account Config

OpenClaw supports Google Chat via a service account key JSON file, enabling bot creation in Google Workspace environments. Alluci has no Google Chat adapter.

- Implement Google Chat API client using service account credentials.
- Handle Chat app webhook verification and message event parsing.

2.9 Channel Health Dashboard

OpenClaw displays a live JSON snapshot of all channel statuses, with a timestamp of the last successful fetch and per-channel error callouts. Alluci exposes no channel health surface.

- Add a /api/channels/status endpoint that aggregates per-adapter health.
- Include: connection state, last message time, error message, account count.

2.10 Multi-Account Support per Channel

OpenClaw tracks multiple accounts per channel (e.g. multiple Telegram bots, multiple Nostr keypairs) and allows per-account routing of cron job deliveries. Alluci is single-account per adapter.

- Add an accounts table in the database linked to each channel type.
- Allow the orchestrator to address a specific account when routing replies.

2.11 Channel Enable / Disable Toggle

OpenClaw allows each channel to be independently enabled or disabled at runtime without restarting the gateway. Alluci loads all adapters on startup and has no runtime toggle.

- Add an enabled flag per channel in the config.
- Implement a hot-reload mechanism that starts or stops the adapter when the flag changes.

2.12 Channel Ordering & Metadata

OpenClaw sorts channel cards — enabled first, then disabled — according to a channelMeta order returned by the gateway. Alluci has no concept of channel ordering or metadata.

- Return a channelMeta array from the gateway hello response.
- Include per-channel icon, display label, and preferred display order.

3 Scheduling & Automation (Cron Engine)

OpenClaw ships a complete cron engine with three schedule types, per-job execution overrides, delivery routing, and a run-history log. Alluci has no scheduling system.

3.1 Three Schedule Types

OpenClaw supports: interval (every N minutes/hours/days), cron-expression (standard 5-field cron syntax), and run-at-date (one-shot ISO datetime). Alluci has no equivalent.

- Implement a CronEngine class backed by a database table (cron_jobs).
- Use a tick loop (setInterval or APScheduler) to evaluate due jobs.
- Parse 5-field cron expressions with timezone support via croniter or croner.

3.2 Per-Job Execution Overrides

Each cron job in OpenClaw can specify: agent ID, model override, thinking level override, and payload type (post-message / run-task / none). Alluci would need the orchestrator to accept per-invocation overrides.

- Add optional model and thinking_level fields to the cron job record.
- Pass overrides into the orchestrator's run_session() entry point.

3.3 Delivery Routing

OpenClaw cron jobs can target a specific channel, account, and recipient for result delivery. Three delivery modes exist: announce-summary, post-transcript, and none. Alluci has no delivery routing.

- Add delivery_channel, delivery_account, delivery_to, delivery_mode fields to cron_jobs.
- After each job run, call the appropriate channel adapter's send() method if delivery_mode is not none.

3.4 Run History Log

OpenClaw stores a run log per job: timestamp, status (ok/error/skipped), duration, delivery status, and the full output text. Alluci has no run history.

- Create a cron_runs table: job_id, started_at, finished_at, status, delivery_status, log_text.
- Write a run record after every job execution.

3.5 Run Log Filtering

OpenClaw's run log supports multi-dimensional filtering: scope (this-job / all-jobs), run status, delivery status, free-text search, and sort direction. Alluci would need a query API for run history.

- Add a /api/cron/runs endpoint with query parameters for all filter dimensions.

3.6 Job Clone Action

OpenClaw allows any existing job to be cloned (duplicated) with a single button click, pre-populating the create form with the job's settings. Alluci would need this as a quick-authoring affordance.

- Implement a POST /api/cron/jobs/:id/clone endpoint.

3.7 Force-Run & Due-Run Modes

OpenClaw supports two manual run modes: force (runs regardless of schedule) and due (runs only if the job is due). Both are available from the UI's Run Now button. Alluci has no manual run capability.

- Add a POST /api/cron/jobs/:id/run endpoint with a mode query parameter.

3.8 Context Reset Flag

OpenClaw cron jobs have a context-reset flag that clears the session context before each scheduled run, preventing context bleed between runs. Alluci's orchestrator has no session reset primitive.

- Add a reset_context boolean to cron_jobs.
- When true, call session.clear_context() before invoking the agent.

4 Usage & Cost Analytics

OpenClaw tracks token usage and cost per session, per day, and per provider, and surfaces this data through a rich analytics dashboard. Alluci's AuditLedger records messages but tracks no token counts or costs.

4.1 Per-Turn Token Tracking

OpenClaw stores input tokens, output tokens, cache-read tokens, and cache-write tokens for every LLM call. Alluci logs requests but captures no token metadata from provider responses.

- Extract token usage from every LLM response (usage field in OpenAI-compatible responses).
- Persist to a usage_log table: session_key, model, provider, input_tokens, output_tokens, cache_read, cache_write, timestamp.

4.2 Per-Model Cost Calculation

OpenClaw maintains a cost table (price per 1M tokens, per model) and multiplies token counts to derive a dollar cost. Alluci has no cost model.

- Create a model_pricing table: model_id, input_price_per_1m, output_price_per_1m, cache_read_price, cache_write_price.
- Add a missing_cost_entries counter for models not in the pricing table.

4.3 Date-Range Analytics API

OpenClaw's usage.sessions RPC accepts start_date and end_date parameters and returns aggregate totals for that window. Alluci has no date-range aggregation endpoint.

- Add a GET /api/usage/sessions endpoint with start, end, and limit query parameters.
- Return: sessions list with per-session aggregates + a totals object.

4.4 Daily Cost Time Series

OpenClaw returns a per-day cost/token breakdown (CostUsageDailyEntry array) for chart rendering. Alluci has no daily rollup.

- Add a GET /api/usage/daily endpoint that returns daily aggregates.

4.5 Per-Session Time Series

OpenClaw can return a per-turn time series (cumulative or per-turn tokens/cost) for a selected session. Alluci has no per-turn time series.

- Add a GET /api/usage/sessions/:key/timeseries endpoint.

4.6 Session-Level Message Log

OpenClaw exposes a message-level log per session: role, content snippet, tool name, timestamp. Alluci's AuditLedger stores messages but has no query API.

- Add a GET /api/sessions/:key/log endpoint with role and tool_name filters.

4.7 CSV Export

OpenClaw provides two one-click CSV exports: sessions summary and daily usage. Alluci has no export capability.

- Add GET /api/usage/export/sessions.csv and /api/usage/export/daily.csv endpoints.

4.8 Missing Cost Warnings

OpenClaw tracks when usage records lack a cost entry (model not in pricing table) and surfaces a warning in the dashboard. Alluci needs this to detect pricing gaps.

- Return a missing_cost_entries count in the aggregation response.
- Log a warning when a model's pricing data is absent.

4.9 Session Token Limit Warning

OpenClaw warns when the 1,000-session query cap is reached, so the user knows the view is truncated. Alluci has no session query limits or warnings.

- Return a limit_reached boolean from the usage sessions endpoint when results are capped.

5 Administration & Control Dashboard

OpenClaw's control UI is a 13-tab web dashboard served over a persistent WebSocket connection. Alluci has no admin dashboard. The closest equivalent is direct API access.

5.1 WebSocket JSON-RPC Gateway

OpenClaw wraps all admin operations in a JSON-RPC 2.0 WebSocket protocol with a hello handshake, event push subscriptions, and method-based RPC calls. Alluci uses plain HTTP REST with no persistent connection.

- Add a /ws WebSocket endpoint to the FastAPI backend.
- Implement a JSON-RPC 2.0 dispatcher over the WebSocket connection.
- Add server-side event push for: stream tokens, turn completion, system health changes.

5.2 Live Connection Status & Uptime

OpenClaw's Overview tab shows gateway uptime, tick interval, auth mode, and connection state with a pulsing green/amber indicator. Alluci has no live health surface.

- Add a system.status RPC method returning: uptime_ms, active_sessions, memory_usage_mb, auth_mode.
- Add a system.health RPC method returning per-subsystem health checks (database, vault, model_router).

5.3 Presence / Instances Panel

OpenClaw tracks all connected clients and nodes as presence beacons (5-minute TTL) and displays them in the Instances tab. Alluci has no presence system.

- Implement a system-presence beacon: each connected client sends a heartbeat every 60s.
- Store beacons in Redis or SQLite with a TTL. Return them via system.presence RPC.

5.4 Per-Session Parameter Overrides

OpenClaw's Sessions tab allows per-session overrides for model, thinking level, verbose level, and reasoning level, all applied via a sessions.patch RPC call. Alluci has no per-session override mechanism.

- Add a sessions.patch endpoint accepting model, thinking_level, verbose_level, reasoning_level fields.
- Store overrides in a session_config table keyed by session_key.

5.5 Session Custom Labels

OpenClaw allows each session to be given a human-readable label, editable inline in the Sessions table. Alluci sessions are identified only by their key.

- Add a label column to the sessions table.
- Expose label in the sessions list response and accept it in sessions.patch.

5.6 Exec Approval Modal

OpenClaw shows a global modal overlay when the agent requests permission to execute a shell command. The user can Deny, Allow Once, or Allow Always. Alluci's SEMI_AUTONOMOUS mode uses a content-length whitelist — there is no interactive approval flow.

- Add an exec.approval WebSocket event pushed to connected UI clients when a tool call requires approval.
- Implement exec.allow(requestId, {persist}) and exec.deny(requestId) RPC methods.
- Store persistent allow policies in a exec_policies table.

5.7 Self-Update Mechanism

OpenClaw has a system.update RPC that runs the self-update process and a sticky update-available banner shown when a newer version is detected. Alluci has no update mechanism.

- Implement a background version check against a release feed.
- Add a system.update RPC method that pulls the latest release and restarts the service.

5.8 Debug / RPC Console

OpenClaw's Debug tab includes a manual RPC console where any JSON-RPC method can be called with arbitrary params. This is invaluable for troubleshooting. Alluci has no equivalent diagnostic tool.

- Expose all RPC methods with their parameter schemas in a methods.list endpoint.
- Build a simple admin console (even a CLI script) that sends raw RPC calls and prints results.

5.9 Live Event Log

OpenClaw's Debug tab maintains a rolling live feed of all WebSocket events received since the UI connected. Each entry shows event name, timestamp, and payload. Alluci has structured logging but no live event feed.

- Add an event_log buffer (capped at 500 entries) in the WebSocket session.
- Push all internal events to connected debug clients.

5.10 Security Audit Summary

OpenClaw's system.status response includes a securityAudit object with critical and warn counters. The Debug tab renders a colour-coded callout and suggests the CLI command openclaw security audit --deep. Alluci has no security audit surface.

- Add a security_audit() function that checks: default credentials, open CORS policies, unencrypted secrets, weak auth modes.
- Return critical/warn/info counts in the system.status response.

5.11 Onboarding Mode

OpenClaw activates a guided onboarding flow on first launch (?onboarding=1 URL parameter). Alluci has no onboarding experience — new users face a blank setup screen.

- Add a first_run flag to the config. When true, redirect the UI to an onboarding wizard.
- The wizard should cover: gateway URL, auth token, default session key, first channel setup.

6 Skill / Plugin Registry

OpenClaw has a structured skill registry with grouping, status reporting, missing-dependency detection, one-click installation, and API key management. Alluci references a skill system in its files but has no registry, no dependency checking, and no installation flow.

6.1 Skill Status Reporting

OpenClaw's skills.status RPC returns a full SkillStatusReport: for each skill, name, description, source (managed / workspace / built-in / bundled), emoji, enabled flag, missing binaries, missing API keys, and install descriptors. Alluci has no equivalent report.

- Create a SkillRegistry class that loads skill manifests from a skills/ directory.
- Each manifest is a YAML/JSON file with: name, description, source, emoji,

requires_bins[], requires_keys[], install[].

- Add a GET /api/skills/status endpoint that returns the full report.

6.2 Missing Dependency Detection

OpenClaw checks whether required binaries (e.g. ffmpeg, imagemagick) are available on PATH for each skill. Missing bins are surfaced as a warning chip on the skill card. Alluci has no dependency checking.

- In SkillRegistry.reload(), run shutil.which(bin) for each requires_bins entry.
- Include missing_bins and missing_keys in the skill status response.

6.3 One-Click Skill Installation

OpenClaw's skill card shows an Install button when a skill has install descriptors and missing binaries. The install process runs in the background and reports progress. Alluci has no installation flow.

- Add a POST /api/skills/:key/install endpoint that runs the skill's install script.
- Stream install progress via WebSocket event skill.install.progress.

6.4 API Key Management per Skill

OpenClaw shows an inline API key input for skills that require external API credentials. The key is saved securely (not returned in the status report). Alluci has no per-skill key storage.

- Store per-skill API keys in VaultManager (not in the skill manifest).
- Add a POST /api/skills/:key/apikey endpoint.
- Return apiKey.required: true and apiKey.isSet: bool in the status report, never the key itself.

6.5 Skill Grouping

OpenClaw groups skills into four categories: managed, workspace, built-in, and bundled — each with different collapse/expand defaults. Alluci has no skill categorisation.

- Add a source field to the skill manifest: one of managed | workspace | built-in | bundled.
- Return skills grouped by source in the status report.

6.6 Bulk Skill Actions (Agent-Scoped)

OpenClaw supports Clear (reset to defaults) and Disable All bulk actions on the per-agent skill panel. Alluci has no bulk skill management.

- Add POST /api/agents/:id/skills/clear and POST /api/agents/:id/skills/disable-all

endpoints.

7 Device & Node Management

OpenClaw implements a full device pairing and node-binding system for distributed execution across multiple machines. Alluci uses a WebAuthn stub that currently returns success unconditionally.

7.1 Signed Device Identity

OpenClaw generates a device identity (public/private keypair) on the client that is signed during the hello handshake. The gateway verifies the signature and approves or rejects based on a devices list. Alluci's WebAuthn verify endpoint is a stub returning unconditional success.

- Generate a persistent device keypair on first connection (stored in localStorage or a secure enclave).
- Sign the hello params with the device private key.
- On the gateway, verify the signature against the device's stored public key.
- Implement devices.list, devices.approve, and devices.reject RPC methods.

7.2 Device Approval Workflow

OpenClaw shows pending pairing requests as callouts in the Nodes tab, with per-request Approve and Reject buttons. Alluci has no approval workflow.

- Add a pending_devices table that stores pairing requests (deviceId, publicKey, timestamp, remoteIp).
- Push a devices.pairing-request event to all admin UI connections when a new request arrives.

7.3 Device Token Rotation

OpenClaw allows rotating the auth token for a paired device without revoking it (devices.rotate RPC). Alluci has no token rotation.

- Implement a devices.rotate endpoint that generates a new token for the device without clearing its approval status.

7.4 Node Capability & Command Metadata

OpenClaw displays each node's caps[] and commands[] arrays (capped at 12 and 8 respectively for display). Alluci has no capability metadata system.

- Add a capabilities field to the presence beacon payload: an array of string capability identifiers (e.g. system.run, browser.screenshot).
- Include caps and commands in the system-presence response so the admin UI can filter nodes by capability.

7.5 Per-Agent Node Binding

OpenClaw allows each agent's shell commands to be dispatched to a specific node (tools.exec.node), with a global default and per-agent overrides. Alluci dispatches all commands locally.

- Add an exec.node field to both the global config and per-agent config.
- In the tool execution path, resolve the target node from the session's agent config before running the command.

8 Configuration Management

OpenClaw provides a schema-driven live config editor with form mode, raw JSON mode, hot-apply, and field-level search. Alluci reads config from environment variables and a static .env file with no live editing capability.

8.1 JSON Schema for Config

OpenClaw exposes the full config schema via a config.schema RPC. The UI uses this to render appropriate widgets for each field without hardcoding form structure. Alluci has no config schema.

- Define a JSON Schema document for the Alluci config (alluci.schema.json).
- Expose it via a GET /api/config/schema endpoint.

8.2 Schema-Driven Form Renderer

OpenClaw's config form uses renderNode() to recursively generate inputs (text, select, checkbox, number, nested object, array list) from the schema. Alluci would need a form renderer to edit config safely.

- Build a form renderer that maps JSON Schema types to HTML inputs.
- Support wildcarded path hints (e.g. agents.list.*.model) for context-sensitive help text.

8.3 Hot-Apply Without Restart

OpenClaw's config.apply RPC applies the current config without restarting the gateway. Alluci

requires a full process restart to pick up config changes.

- Implement a config.reload() method that re-reads the config file and re-initialises mutable services (model router, vault, guardrail).
- Expose via POST /api/config/apply.

8.4 Config Dirty State & Unsaved Change Warnings

OpenClaw tracks whether the current form state differs from the last-saved state and shows a dirty indicator. Alluci has no equivalent.

- Track a config_dirty flag in the admin session. Set it on any form change; clear it on save/reload.
- Return the current saved config hash in the GET /api/config response so clients can detect drift.

9 Log Streaming & Observability

OpenClaw's Logs tab streams and filters the gateway's JSONL log file with level chips, text search, auto-follow scrolling, and one-click export. Alluci uses Python's structlog for structured logging but has no log-viewing interface or streaming endpoint.

9.1 JSONL Log Streaming Endpoint

OpenClaw reads the log file in chunks and streams entries over the WebSocket (logs.read RPC). Alluci has no log streaming endpoint.

- Add a GET /api/logs endpoint that returns the last N JSONL lines.
- Add a WebSocket log subscription event: logs.entry, pushed as new lines are written.

9.2 Log Level Filtering

OpenClaw supports 6-level log filtering (trace / debug / info / warn / error / fatal) with colour-coded chips. Alluci logs at info/warning/error but provides no filter API.

- Accept a levels[] query parameter on the log endpoint.
- Apply level filtering on the server side before streaming.

9.3 Auto-Follow with Scroll Detection

OpenClaw's log view auto-scrolls to new entries until the user scrolls up, at which point it pauses. Alluci has no auto-follow behaviour.

- Add a follow=true query parameter to the WebSocket log subscription.
- The client controls auto-scroll; the server only needs to push new entries promptly.

9.4 Log Export

OpenClaw exports filtered or all-visible log entries as a plain text file with a dynamic filename. Alluci has no log export capability.

- Add a GET /api/logs/export endpoint that returns filtered log lines as a downloadable .txt file.

10 Chat Interface UX

Alluci has a functional chat via its WebSocket endpoint. OpenClaw's chat UI adds several production-polish features that meaningfully improve usability.

10.1 Image Paste Attachments

OpenClaw handles paste events in the compose area, extracts image clipboard items, and converts them to base64 data-URLs for sending with the next message. Alluci's chat accepts only text.

- Add a paste handler to the compose input that reads image items from the clipboard.
- Convert to base64 and include as a content block in the outbound message.

10.2 Message Queue (Send While Streaming)

OpenClaw queues messages sent while a streaming response is in progress, then dispatches them in order when the stream completes. Alluci drops or blocks new sends during streaming.

- Implement a client-side message queue; when a send is attempted while busy, push to the queue.
- After each stream completion event, dequeue and send the next item.

10.3 Context Compaction Indicator

OpenClaw shows a visible divider in the message thread when the context window has been compacted, and displays a live compactionStatus banner above the compose area during compaction. Alluci performs compaction silently.

- Tag compaction events in the message stream with a __alluci.kind = 'compaction' marker.
- Push a compaction.status event to the client before and after compaction.

- Render a thread divider at the compaction point.

10.4 Model Fallback Indicator

OpenClaw shows a fallbackStatus banner when the primary model has failed and a fallback model is in use. Alluci performs silent failover.

- Push a model.fallback WebSocket event when the router switches to a fallback model.
- Include the fallback model name in the event payload for display.

10.5 Resizable Split-Pane Sidebar

OpenClaw's chat can open a draggable split-pane sidebar to display Markdown artifacts alongside the thread. Alluci has no artifact sidebar.

- Add a sidebar_content field to the chat state that, when non-null, activates a split-pane layout.
- Implement a resizable divider that stores the split ratio in localStorage.

10.6 Focus Mode

OpenClaw has a Focus Mode button in the chat header that collapses the navigation sidebar and maximises the chat thread. Alluci has no focus mode.

- Add a focus_mode boolean to the UI state toggled by a keyboard shortcut or button.
- When active, collapse the nav sidebar and hide non-essential chrome.

10.7 Configurable Assistant Avatar & Name

OpenClaw reads assistantName and assistantAvatar (URL or data-URL) from the agent identity config and renders them in the chat thread. Alluci uses a generic system label.

- Expose assistant name, emoji, and avatar URL from the agent's identity endpoint.
- Use the agent identity when rendering assistant messages in the chat.

11 Security & Authentication Infrastructure

Several of Alluci's security components are stubs or have known vulnerabilities identified in the audit report. OpenClaw provides reference implementations for each.

11.1 Real WebAuthn Verification

OpenClaw's device identity flow verifies cryptographic signatures on the server side. Alluci's /auth/webauthn/verify endpoint returns SUCCESS unconditionally — it is a stub. This is a critical vulnerability.

- Implement actual WebAuthn server-side verification using py_webauthn.
- Store credential IDs and public keys in the database on registration.
- Verify signatures on each authentication attempt.

11.2 Auth Mode Negotiation

OpenClaw supports four auth modes: none, token, password, and trusted-proxy. The mode is declared in the hello handshake so clients can adapt their credential presentation. Alluci uses a single token-only mode with no negotiation protocol.

- Add an auth_mode field to the gateway config.
- Return the active auth_mode in the hello/health response so clients know which credentials to present.

11.3 Timing-Safe Secret Comparison

Alluci's master key comparison uses Python's == operator, which is vulnerable to timing attacks (BUG-05 in the audit). OpenClaw uses constant-time comparison throughout.

- Replace all secret comparison operations with hmac.compare_digest().
- This is a one-line fix per occurrence — fix immediately.

11.4 BioVault Token Encryption

Alluci's BioVault state tokens are base64-encoded JSON, trivially decodable. OpenClaw stores all sensitive state in Fernet-encrypted blobs. Alluci should apply the same.

- Encrypt BioVault state tokens using VaultManager.store_secret() instead of base64 encoding.
- The decryption key should be the session's Fernet key, not a hardcoded value.

11.5 Trusted Proxy Mode

OpenClaw supports a trusted-proxy auth mode for deployments behind Tailscale or a reverse proxy with device identity, replacing token auth. Alluci has no proxy-aware auth mode.

- Add a TRUSTED_PROXY_HEADER env var. When set, accept identity from the X-Forwarded-User header instead of a token.
- Document the Tailscale Serve setup for users who want to avoid token auth.

12 Consolidated Priority Matrix

All 36 gap items from Sections 2–11 consolidated into a single ranked table. Priority is defined as: Critical = blocks security or core functionality; High = materially limits production viability; Medium = significantly improves user experience or observability; Low = nice to have.

1	Real WebAuthn verification (BUG-02)	Security	Critical	S	Unconditional success is an open door
2	Timing-safe secret comparison (BUG-05)	Security	Critical	XS	One-line fix, high exploitability
3	BioVault token encryption	Security	Critical	S	Sensitive state exposed in plaintext
4	Working Telegram adapter	Channels	Critical	M	Most common bot deployment target
5	Working WhatsApp adapter + QR pairing	Channels	Critical	L	Highest consumer reach
6	Cron engine (3 schedule types + DB persistence)	Scheduling	Critical	L	Enables all autonomous automation
7	Working Discord adapter	Channels	High	M	Developer/ community audience
8	Working Slack adapter (OAuth)	Channels	High	M	Enterprise B2B use case
9	Per-turn token tracking + cost calculation	Analytics	High	M	Needed for cost visibility
10	WebSocket JSON-RPC gateway	Admin	High	L	Foundation for all live admin features
11	Exec approval modal + persist policy	Admin	High	M	Replaces dangerously permissive whitelist

12	Delivery routing for cron jobs	Scheduling	High	M	Connects scheduler to channels
13	Cron run history log	Scheduling	High	S	Essential for debugging automated runs
14	Date-range analytics API	Analytics	High	M	Required for cost management
15	Skill status report + dependency check	Skills	High	M	Production-readiness gate
16	Device identity + approval workflow	Devices	High	L	Replaces unconditional WebAuthn stub
17	JSON Schema for config + hot-apply	Config	High	M	Eliminates restart-to-reconfigure cycle
18	JSONL log streaming endpoint	Observability	High	S	Basic operational visibility
19	Image paste attachments in chat	Chat UX	High	S	Expected by users, simple to add
20	Signal adapter (signal-cli)	Channels	Medium	M	Privacy-focused user segment
21	Google Chat adapter	Channels	Medium	M	Enterprise Workspace deployments
22	Per-session time series chart	Analytics	Medium	M	Deep dive into session cost
23	CSV export (sessions + daily)	Analytics	Medium	S	Finance / reporting use case
24	Presence / instances panel	Admin	Medium	M	Multi-node visibility
25	Per-session model overrides	Admin	Medium	S	A/B testing across sessions
26	Security audit summary in status	Admin	Medium	S	Proactive security posture
27	One-click skill install flow	Skills	Medium	M	Reduces setup friction
28	Per-skill API key management	Skills	Medium	S	Secure external service credentials

29	Per-agent node binding	Devices	Medium	M	Required for multi-machine deployments
30	Context compaction thread divider	Chat UX	Medium	S	Transparency for long sessions
31	Resizable split-pane sidebar	Chat UX	Medium	M	Artifact display alongside chat
32	Message queue (send while streaming)	Chat UX	Medium	S	Prevents message loss during generation
33	Nostr adapter (NIP-01 + NIP-04)	Channels	Low	L	Niche but aligns with sovereignty theme
34	iMessage adapter (macOS only)	Channels	Low	M	Apple ecosystem, limited server deployment
35	Onboarding wizard on first launch	Admin	Low	M	Nice to have, not blocking
36	Log level filtering + auto-follow	Observability	Low	S	Quality-of-life for developers

Effort Key

XS	< 1 day	Single-function fix, no new tables or endpoints
S	1–3 days	New endpoint or table, straightforward logic
M	1–2 weeks	New subsystem with moderate integration work
L	2–6 weeks	Major subsystem requiring design, testing, and integration

13 Suggested Implementation Roadmap

The following sprint plan sequentially addresses the gap items, front-loading critical security fixes and foundational infrastructure before layering feature capabilities on top.

Sprint 0 — Security Fixes (1 week)

Immediate action required

These items are bugs or vulnerabilities. They must be fixed before any public deployment or further feature work. All are small, isolated code changes.

- **BUG-05:** Replace == with hmac.compare_digest() for all secret comparisons. ~30 minutes.
- **BUG-02:** Implement real WebAuthn verification using py_webauthn. ~1 day.
- **BUG-04:** Add await to vault.retrieve_secret() calls. ~30 minutes.
- **BUG-03:** Fix NameError in /ready endpoint. ~15 minutes.
- **BUG-01:** Remove duplicate exception handler registration. ~15 minutes.
- **BioVault tokens:** Encrypt using VaultManager instead of base64. ~2 hours.

Sprint 1 — Foundation Infrastructure (2–3 weeks)

Build the WebSocket gateway and database schema that all subsequent features depend on.

- WebSocket JSON-RPC gateway (Section 5.1)
- Per-turn token tracking + cost tables (Section 4.1–4.2)
- Cron engine with DB persistence (Section 3.1)
- JSONL log streaming endpoint (Section 9.1)
- JSON Schema for config + hot-apply (Section 8.1–8.3)

Sprint 2 — First Two Channels (2–3 weeks)

Implement the highest-priority channel adapters end-to-end, including health reporting and multi-account support.

- Telegram bot adapter: token validation, webhook, inbound parsing, account tracking (Section 2.2)
- WhatsApp adapter: QR pairing flow, session persistence, send/receive (Section 2.1)
- Channel health dashboard endpoint (Section 2.9)
- Channel enable/disable runtime toggle (Section 2.11)

Sprint 3 — Automation & Analytics (2 weeks)

Complete the cron engine and wire up the analytics API.

- Cron job delivery routing to channels (Section 3.3)
- Cron run history log (Section 3.4–3.5)
- Date-range analytics API + daily rollup (Section 4.3–4.4)
- CSV export (Section 4.7)

- Exec approval modal + persist policy (Section 5.6)

Sprint 4 — Remaining Channels + Device Management (2–3 weeks)

- Discord adapter (Section 2.3)
- Slack OAuth adapter (Section 2.4)
- Device identity + pairing approval workflow (Section 7.1–7.2)
- Per-agent node binding (Section 7.5)
- Skill status report + one-click install (Section 6.1–6.3)

Sprint 5 — Polish & Observability (1–2 weeks)

- Chat: image paste, message queue, compaction divider, focus mode (Sections 10.1–10.6)
- Per-session model overrides (Section 5.4)
- Session custom labels (Section 5.5)
- Security audit summary in status (Section 5.10)
- Log level filtering + export (Sections 9.2, 9.4)
- Onboarding wizard (Section 5.11)

Sprint 6+ — Lower-Priority Items

- Signal adapter (Section 2.5)
- Google Chat adapter (Section 2.8)
- Nostr adapter (Section 2.7)
- iMessage macOS adapter (Section 2.6)
- Per-session time series chart + session log API (Section 4.5–4.6)
- Multi-account support per channel (Section 2.10)
- Device token rotation (Section 7.3)
- Node capability metadata (Section 7.4)

Architectural note

The single most impactful architectural change Alluci can make is implementing the WebSocket JSON-RPC gateway (Section 5.1). Every live feature — streaming, event push, exec approval, log tailing, skill installation progress — depends on this foundation. Build it first, and all other features become straightforward to add.